

Project 5: Loan Status Prediction

importing the Dependencies

```
In [1]: import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn.metrics import accuracy_score
```

Data Collection and Processing

```
In [2]: #Loading the dataset to pandas DataFrame
loan_dataset=pd.read_csv(r'D:\Projects\archive (4)/loan_data.csv')
```

```
In [3]: type(loan_dataset)
```

```
Out[3]: pandas.core.frame.DataFrame
```

```
In [4]: #printing the first 5 rows of the dataframe
loan_dataset.head()
```

```
Out[4]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
0	LP001003	Male	Yes	1	Graduate	No	4583
1	LP001005	Male	Yes	0	Graduate	Yes	3000
2	LP001006	Male	Yes	0	Not Graduate	No	2583
3	LP001008	Male	No	0	Graduate	No	6000
4	LP001013	Male	Yes	0	Not Graduate	No	2333

```
In [5]: # number of rows and columns
loan_dataset.shape
```

```
Out[5]: (381, 13)
```

```
In [6]: #statistical measures
loan_dataset.describe()
```

Out[6]:

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
count	381.000000	381.000000	381.000000	370.000000	351.000000
mean	3579.845144	1277.275381	104.986877	340.864865	0.834473
std	1419.813818	2340.818114	28.358464	68.549257	0.369911
min	150.000000	0.000000	9.000000	12.000000	0.000000
25%	2600.000000	0.000000	90.000000	360.000000	1.000000
50%	3333.000000	983.000000	110.000000	360.000000	1.000000
75%	4288.000000	2016.000000	127.000000	360.000000	1.000000
max	9703.000000	33837.000000	150.000000	480.000000	1.000000

In [7]: *#number of missing values in each column*
 loan_dataset.isnull().sum()

Out[7]:

Loan_ID	0
Gender	5
Married	0
Dependents	8
Education	0
Self_Employed	21
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	0
Loan_Amount_Term	11
Credit_History	30
Property_Area	0
Loan_Status	0

dtype: int64

In [8]: *# dropping the missing values*
 loan_dataset=loan_dataset.dropna()

In [9]: *#Label encoding*
 loan_dataset.replace({r"Loan_Status":{'N':0,'Y':1}},inplace=True)

C:\Users\Admin\AppData\Local\Temp\ipykernel_14200\4047168103.py:2: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer\_objects(copy=False)`. To opt-in to the future behavior, set `pd.set\_option('future.no\_silent\_downcasting', True)`
 loan_dataset.replace({r"Loan_Status":{'N':0,'Y':1}},inplace=True)

In [10]: *#printing the first 5 rows of the dataframe*
 loan_dataset.head()

Out[10]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
0	LP001003	Male	Yes	1	Graduate	No	4583
1	LP001005	Male	Yes	0	Graduate	Yes	3000
2	LP001006	Male	Yes	0	Not Graduate	No	2583
3	LP001008	Male	No	0	Graduate	No	6000
4	LP001013	Male	Yes	0	Not Graduate	No	2333

In [11]: *#Dependent column values*
loan_dataset['Dependents'].value_counts()

Out[11]: Dependents
0 194
2 47
1 43
3+ 24
Name: count, dtype: int64

In [12]: *#replacing the value of 3+ to 4*
loan_dataset=loan_dataset.replace(to_replace='3+',value=4)

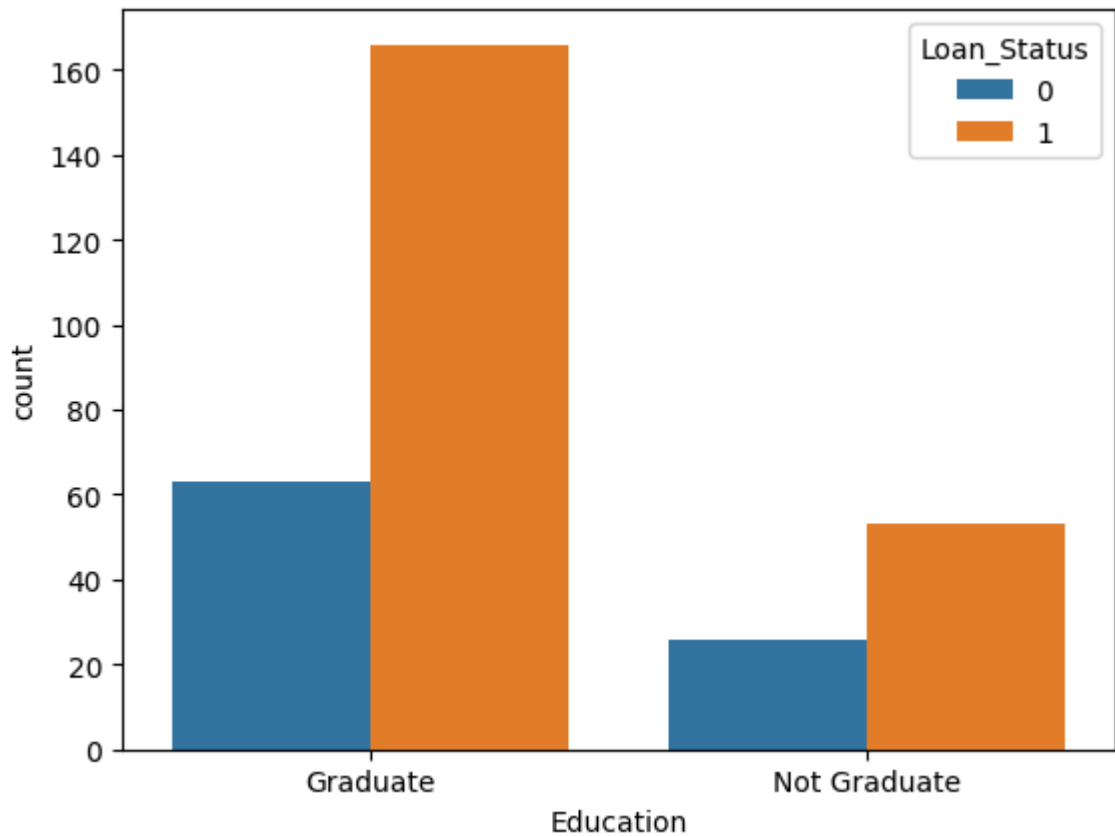
In [13]: *#dependent values*
loan_dataset['Dependents'].value_counts()

Out[13]: Dependents
0 194
2 47
1 43
4 24
Name: count, dtype: int64

Data Visualization

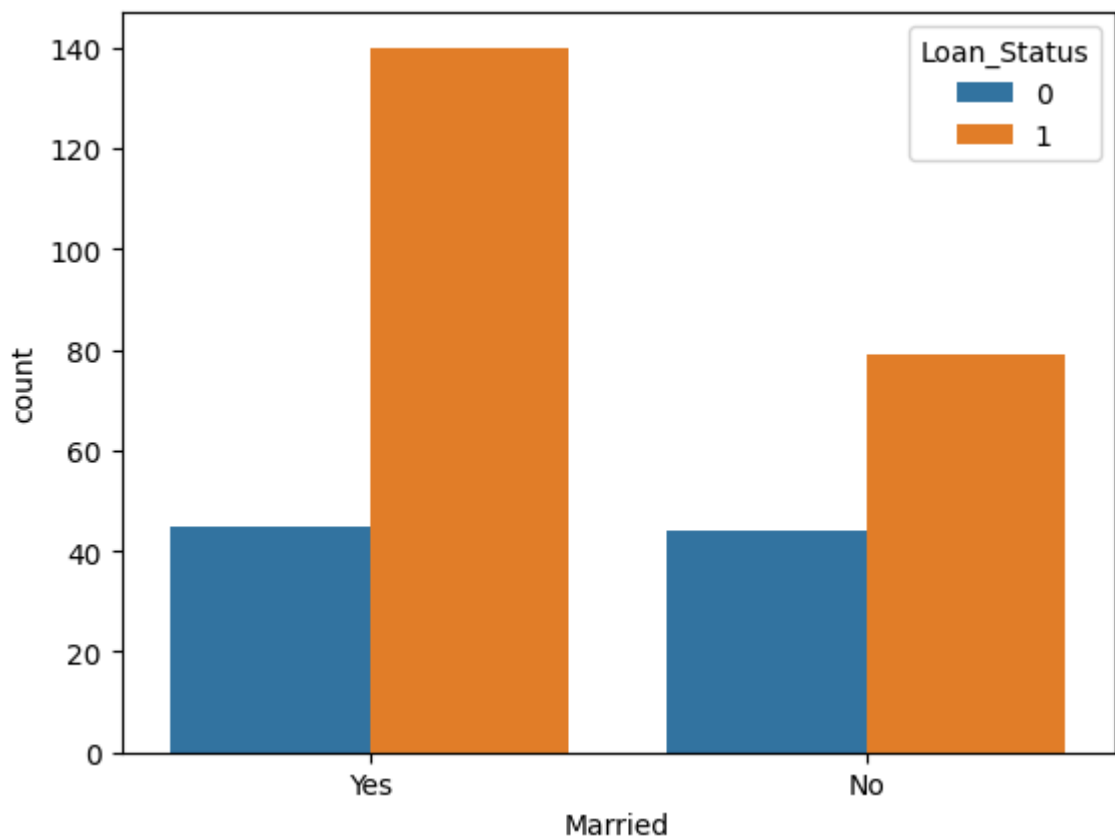
In [14]: *# education & Loan Status*
sns.countplot(x='Education',hue='Loan_Status',data=loan_dataset)

Out[14]: <Axes: xlabel='Education', ylabel='count'>



```
In [15]: #marital status & Loan Status
sns.countplot(x='Married',hue='Loan_Status',data=loan_dataset)
```

```
Out[15]: <Axes: xlabel='Married', ylabel='count'>
```



```
In [16]: # convert categorical columns to numerical values
loan_dataset.replace({'Married':{'No':0,'Yes':1}, 'Gender':{'Male':1,'Female':0}}
```

C:\Users\Admin\AppData\Local\Temp\ipykernel_14200\2300042596.py:2: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer\_objects(copy=False)`. To opt-in to the future behavior, set `pd.set\_option('future.no\_silent\_downcasting', True)`

```
loan_dataset.replace({r'Married': {'No': 0, 'Yes': 1}, 'Gender': {'Male': 1, 'Female': 0}, 'Self_Employed': {'No': 0, 'Yes': 1}, 'property_Area': {'Rural': 0, 'Semiurban': 1, 'Urban': 2}, 'Education': {'Graduate': 1, 'Not Graduate': 0}}, inplace=True)
```

In [17]: `loan_dataset.head()`

Out[17]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
0	LP001003	1	1	1	1	0	4583
1	LP001005	1	1	0	1	1	3000
2	LP001006	1	1	0	0	0	2583
3	LP001008	1	0	0	1	0	6000
4	LP001013	1	1	0	0	0	2333

In [18]: *#separting the data and label*
`x=loan_dataset.drop(columns=['Loan_ID', 'Loan_Status'],axis=1)`
`y=loan_dataset['Loan_Status']`

`print(x)`
`print(y)`

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	\
0	1	1	1	1	0	4583	
1	1	1	0	1	1	3000	
2	1	1	0	0	0	2583	
3	1	0	0	1	0	6000	
4	1	1	0	0	0	2333	
..	...	...	...	...	...	...	
376	1	1	4	1	0	5703	
377	1	1	0	1	0	3232	
378	0	0	0	1	0	2900	
379	1	1	4	1	0	4106	
380	0	0	0	1	1	4583	

	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	\
0	1508.0	128.0	360.0	1.0	
1	0.0	66.0	360.0	1.0	
2	2358.0	120.0	360.0	1.0	
3	0.0	141.0	360.0	1.0	
4	1516.0	95.0	360.0	1.0	
..	...	...	...	...	
376	0.0	128.0	360.0	1.0	
377	1950.0	108.0	360.0	1.0	
378	0.0	71.0	360.0	1.0	
379	0.0	40.0	180.0	1.0	
380	0.0	133.0	360.0	0.0	

	Property_Area
0	Rural
1	Urban
2	Urban
3	Urban
4	Urban
..	...
376	Urban
377	Rural
378	Rural
379	Rural
380	Semiurban

[308 rows x 11 columns]

0	0
1	1
2	1
3	1
4	1
..	
376	1
377	1
378	1
379	1
380	0

Name: Loan_Status, Length: 308, dtype: int64

Train_Test_Split

```
In [19]: from sklearn.preprocessing import LabelEncoder

le=LabelEncoder()
loan_dataset['Property_Area']=le.fit_transform(loan_dataset['Property_Area'])

In [20]: x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.1,stratify=y,rand
print(x.shape,x_train.shape,x_test.shape)

(308, 11) (277, 11) (31, 11)
```

Training the model:

Support Vector Machine Model

```
In [21]: classifier=svm.SVC(kernel='linear')
```

```
In [22]: print(x_train.dtypes)
```

```
Gender          int64
Married         int64
Dependents      object
Education       int64
Self_Employed   int64
ApplicantIncome int64
CoapplicantIncome float64
LoanAmount      float64
Loan_Amount_Term float64
Credit_History  float64
Property_Area   object
dtype: object
```

```
In [23]: from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
for col in x_train.select_dtypes(include='object').columns:
    x_train[col]=le.fit_transform(x_train[col].astype(str))
```

```
In [24]: classifier.fit(x_train,y_train)
```

Out[24]:

▼ SVC ⓘ ?

- Parameters
- Fitted attributes

Model Evaluation

```
In [25]: #Accuracy score on training data
x_train_prediction=classifier.predict(x_train)
training_data_accuracy=accuracy_score(x_train_prediction,y_train)
```

```
In [26]: print('Accuracy on training data:',training_data_accuracy)
```

Accuracy on training data: 0.8303249097472925

```
In [36]: #Accuracy score test data  
x_test_prediction=classifier.predict(x_test)  
test_data_accuracy=accuracy_score(x_test_prediction,y_test)
```

```
In [37]: print('Accuracy on test data:',test_data_accuracy)
```

Accuracy on test data: 0.7580645161290323